

```
x,y = 2,3
x,y = y,x
print(x)
print(y)
```

2. Nedan har vi kod som definierar variablerna x och y med tilldelade värden. Hitta på ett aritmetiskt uttryck i x och y som ger värdet

100.

```
x = 4
y = -3
```

3. Låt

```
s = 'hej'
```

Skriv kod som skriver ut 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' (det vill säga, hej sexton gånger på raken).

4. Låt

```
s = 'hej'
```

Skriv kod som skriver ut 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' genom att bara använda operatörn +, men så få gånger som möjligt.

5. Skriv kod som frågar användaren om dess förnamn och efternamn samt sparar resultatet i två variabler. Skriv sedan ut dessa. Ta inspiration från stycket 'Input' ovan (avsnitt 1.6). Till exempel ska det fungera så här:

```
Vad heter du i förnamn?    # Skriv in ditt förnamn här, tex 'Johan'
Vad heter du i efternamn?  # Skriv in ditt efternamn här, tex 'Jansson'
Hej Johan Jansson
```

6. Skriv kod som frågar användaren om dess ålder (spara i variabeln age) samt favorittal (spara i variabeln num). Vad händer om du adderar dessa variabler? Blir det vad du förväntar dig? Vad drar du för slutsatser om age och num?
7. Gör beräkningen $70 / 1.5$ i Python. Vad är det som kan uppfattas som konstigt med svaret och vad beror det på?

Kapitel 2

Grunder i Python-programmering

2.1 Syntax och semantik

Inom programmering pratar man ofta om *syntax* och *semantik*:

- *Syntax* handlar om regler för *hur* man skriver.
- *Semantik* handlar om *betydelsen* av vad man skriver.

Som ett exempel kan vi tänka på följande mening på Engelska:

Colorless green ideas sleep furiously.

Mening är syntaktiskt korrekt, dvs den innehåller inget *grammatiskt* fel, men *innebörden* är inte något meningsfullt: semantiken är nonsens.

Syntax är speciellt viktigt, och kan vara svårt, när man börjar lära sig att programmera. Precis som i ett vanligt språk så handlar det om att lära sig reglerna för vad som är korrekt. Om man använder Python handlar det till exempel om vad som är en giltig identifierare, var och hur man måste indentera kod-block, hur man skriver if-satser och for-loopar (vi kommer att gå igenom dessa i senare kapitel), och så vidare. Syntax är specifikt för ett visst programmeringsspråk, även om det ofta finns många likheter mellan språk. Python har till exempel många likheter med språket C. Att lära sig syntaxen går relativt snabbt genom övning, och fokus flyttas sedan till semantiken: vad händer i programmet (eller vad *borde* hända i programmet)?

Om du ser någonting i stil med `SyntaxError` i ett felmeddelande när du exekverar Python-kod så betyder det att du har skrivit något litet fel och att Python inte kan förstå din kod. I dessa fall får man ofta ett ganska instruktivt felmeddelande, eftersom Python kan säga till om vad som går fel. Till exempel kan följande felmeddelande fås:

```
3a = 2
^
SyntaxError: invalid syntax
```

Felmeddelandet visar var den ogiltiga syntaxen förekommer: resultatet av ett ogiltigt namn för en identifierare.

Semantiska fel är svårare att hitta, eftersom Python inte säger till om något fel. Allt är giltig kod. Resultatet av semantiska fel är att programmet inte gör vad man förväntar sig. Detta kan undvikas genom att skriva

välorganiserad, väldokumenterad och välstrukturerad kod, dvs att skriva “bra” kod. Om det är tydligt vad man *vill* att koden ska göra när någon (eller man själv) läser koden, blir det mycket lättare att hitta semantiska fel.

```
x = 7
y = 5
x_minus_y = y - x
print("x minus y = ", x_minus_y)
```

Här har vi ett exempel på ett semantiskt fel. Koden kör från början till slut utan problem men resultat blir fel eftersom vi råkar beräkna $y - x$ istället för $x - y$ på rad 3. Med det tydliga variabelnamnet `x_minus_y` är det ganska lätt att hitta vart felet uppstod när man kollar igenom koden.

2.2 Datatyper

Alla värden i Python har en typ. Vi har redan använt oss av olika typer när vi till exempel tilldelat variabler värden som heltal, flyttal och strängar. De vanligaste typerna i Python som vi kommer att använda är:

- `str` – strängar, dvs bokstäver (eller generellt tecken) inom `'` eller `"`
- `int` – heltal, av obegränsad storlek
 - I de flesta programmeringsspråk är heltal begränsade till 32 bitar, vilket ger möjliga värden från -2^{32} till $2^{31} - 1$.
- `float` – flyttal, en approximation av reella tal
 - För approximering av reella tal i intervallet $[-10^{308}, 10^{308}]$, med 15 siffrors decimal precision.
 - En float kodas med 64 bitar, varav en bit ger tecken (negativt/positivt), 11 används för exponenten x och 52 för “mantissan” m för att ge talet $m2^x$.
 - Detaljer kan fås genom `import sys` och objektet `sys.float_info`
 - Varför är resultatet inte alltid exakt vad man förväntar sig? Jo, Python representerar flyttal genom bråk i bas 2. För detaljer se: <https://docs.python.org/3/tutorial/floatingpoint.html>
 - Generellt gäller att det inte går att representera *alla* reella tal *exakt* på en dator, vilken talrepresentation som än väljs. Vissa reella tal kan representeras exakt medan andra får bli en approximation: fler bitar resulterar i en bättre approximation.
- `complex` – skrivs `1+2j`
 - Representeras med två flyttal
- `bool` – Booleska värden (dvs ‘sant’ eller ‘falskt’), representeras av de reserverade orden `True` och `False`.
- `NoneType` – för värdet `None`

Man kan i Python se vilken typ en variabel eller literal har genom att använda sig av `type`:

```
my_int = 1
my_float = 1.0
my_string = "1"
print(type(my_int))
print(type(my_float))
print(type(my_string))
print(type('hello'))
```

```
int
float
```

```
str
str
```

2.2.1 Typomvandling

Man kan omvandla en typ till en annan, men det finns regler om hur specifika typer omvandlas och till vilka typer de kan omvandlas. Ofta behöver man inte ens tänka på att en typ omvandlas, utan det funkar ganska mycket som man rent intuitivt tänker sig. En sådan typomvandling kallas *implicit*. Till exempel är det sällan ett problem att blanda numeriska typer i ett uttryck:

```
my_int = 1
my_float = 1.0
c = my_int + my_float
print(type(c))
```

```
float
```

Här omvandlas alltså värdet i `my_int` till ett flyttal som sedan adderas till flyttalet som finns i `my_float`, och resulterar i ett flyttal `c`.

Som visats tidigare så finns det situationer där en sådan konvertering inte fungerar. Skulle man till exempel utvärdera uttrycket `my_int + my_string` skulle man få ett `TypeError`. Man kan då använda sig av *explicit konvertering*, vilket man gör genom att ge typen man vill omvandla ett värde till värdet som argument. Om vi skulle vilja typomvandla `x`, så skulle vi kunna göra:

```
str(x)
int(x)
float(x)
complex(x)
bool(x)
```

Vi kan nu använda oss av explicit konvertering för att göra:

```
c = my_int + int(my_string)
print(type(c))
```

```
int
```

I detta fall kan värdet i `my_string` omvandlas till en `int` för att sedan adderas till `my_int`.

Varning: explicit typomvandling funkar inte alltid “magiskt”:

```
int("hello!")
```

```
ValueError: invalid literal for int() with base 10: 'hello!'
```

Här vet inte Python hur `'hello!'` ska omvandlas till ett heltal.

Explicit konvertering är ofta viktigt när man använder `input()`, som vid lyckad inläsning ger typen `str`. I vissa fall vill man inte att ens variabel ska vara en `str`, som nedan:

```
age = int(input("Din ålder: "))
added_age = age + 10 # would not work if age was of type str
print(added_age)
```

2.3 Booleska uttryck i Python

En operator är en symbol som använd på en eller flera värden, operander, ger ett nytt värde. Vi har tidigare sett exempel på typiska aritmetiska operatorer (+, -, *, /, ...). Resultatets typ beror av operandernas typer. Booleska uttryck använder sig av operatorer som resulterar i att en bool returneras, dvs True eller False. Sådana uttryck är väldigt användbara när man ska utvärdera om något är sant eller falsk givet ett test.

Det är värt att påpeka att det *mesta* i Python anses vara *sant*. Vi kan undersöka detta genom att använda typomvandling till bool:

```
bool(3)
bool(-427.2)
bool('hello')
```

```
True
True
True
```

Värden som anses vara *falska* är typiskt “tomma” värden, vad de än har för typ:

```
bool(0)
bool(0.0)
bool('')
bool(None)
```

```
False
False
False
False
```

2.3.1 Jämförelseoperatorer

Dessa operationer jämför indata genom följande relationer:

- Likhet: `x == y`
- Olikhet: `x != y`
- Strikt större än: `x > y`
- Strikt mindre än: `x < y`
- Större än eller lika: `x >= y`
- Mindre än eller lika: `x <= y`

Dessa fungerar som man förväntat för siffror (experimentera!), men som många operatorer i Python är det inte *endast* int eller float man kan jämföra. Till exempel:

```
print('hello shark' < 'hello shark')
print('hello shark' < 'hello shork')
print('hello shark' < 'hello')
print('hello shark' < 'Hello shork')
```

```
False
True
False
False
```

Sök på internet om hur Python jämför strängar!

2.3.2 Logiska operatorer

Python använder sig av de logiska operatorerna and, or och not.

- Är x *och* y sant?: x and y
x and y är True endast om både x och y är True.
- Är x *eller* y sant?: x or y
x or y är True endast om någon av x och y är True. Notera att att x or y alltså blir True även när både x och y är sanna.
- Negera ett värde: not x
not x är True om x är False och False annars.

Detta kan skrivas med sanningsvärdestabeller. Vi kan till exempel använda oss av logiska operatorer för att definiera ett uttryck som är ett exklusiv-eller, xor, som utvärderas till sant om exakt en av x eller y är sann:

```
x = True
y = False
xor_result = (not ((x and y) or (not x and not y)))
print(xor_result)
```

```
True
```

Python behandlar and och or lite speciellt om (båda) operanderna inte är av typen bool: en and/or-sats evalueras då inte till bool. En and-sats ger tillbaka den *andra* operanden om båda operanderna är sanna, men ger tillbaka värdet av den *falska* operanden om uttrycket innehåller en sådan. En or-sats ger tillbaka det *första* värdet om första operanden är sann, annars värdet av andra operanden. Några exempel:

```
(42 and 'hello')
(42 and '')
(False and '')
(42.7 or 13)
('42' or None)
(False or 0)
```

```
'hello'
''
False
42.7
'42'
0
```

De här evalueringsreglerna kan förefalla konstiga, men har att göra med hur mycket av satsen som behöver utvärderas för att garanterat veta resultatet av hela uttrycket. Om första operanden till and är falsk, då behöver man inte beräkna värdet på det andra operanden; oavsett om den evaluerar till True eller False så ska resultatet motsvara False. Tekniken kallas ibland “kortslutning” (eng: *short circuiting*), eller mer generellt “lat evaluering” (eftersom det kan ses som “lathet” att inte beräkna allting). Syftet är förstås att få snabbare program genom att inte beräkna uttryck som inte behövs.

2.4 Tilldelningsoperatorer

Vi har redan sett att `=` används för tilldelning. En smidig funktionalitet i Python är att man kan kombinera detta med aritmetiska operatorer och få så kallade "tilldelningsoperatorer":

```
x += 3    # x = x + 3
x -= 3    # x = x - 3
x *= 3    # x = x * 3
x /= 3    # x = x / 3
x %= 3    # x = x % 3
x //= 3   # x = x // 3
x **= 3   # x = x ** 3
```

2.5 Funktioner

I och med att ens program blir större och kanske mer komplicerat, så märker man snabbt att det finns ett behov att kunna strukturera upp sin kod. Det kan man börja göra med hjälp av *funktioner*. Vi har redan använt oss av flera av Pythons inbyggda funktioner, till exempel `print`. Funktioner inom programmering har, som namnet antyder, likheter med matematiska funktioner: de tar ett eller flera argument och ger tillbaka ett returvärde. Inom programmering *behöver* dock inte en funktion ta några argument eller lämna tillbaka ett värde.

En av fördelarna med att använda funktioner är att du kan återanvända delar av koden istället för att upprepa samma kod på flera ställen. Detta gör ofta koden mer lättläst. Det gör också koden lättare att uppdatera då du bara behöver ändra på en plats istället för att ändra på alla platser där den upprepade koden var.

Funktioner delar också upp ditt program i mindre delar. Fördelarna med denna uppdelning är bland annat:

- Det tydliggör beroenden i koden: "indata" som argument, resultat efter **return**. Det här påminner om en funktion i matematiken.
- Det ger naturliga stycken att dokumentera.
- Det ger naturliga stycken att testa för att försäkra sig att koden fungerar som den ska.

Ett program uppdelat i många mindre funktioner som fungerar fristående från varandra kallas *modulärt* medan ett program där all kod körs som ett stort stycke kallas *monolitiskt*. Monolitiska program kan vara lättare att skriva i stunden men är både svårare att underhålla och att läsa och förstå.

Funktioner är därför ett mycket viktigt verktyg för att skriva bra kod.

Innan man kan använda en funktion, så måste funktionen *definieras*. Det gör man genom att använda sig av det reserverade ordet **def** för att ge funktionen ett namn (identifierare) och specificera argument. Nedan är ett exempel på en enkel funktion `f` som tar ett värde `x` som argument och adderar värdet till sig själv:

```
def f(x):
    return x + x
```

Denna funktion skulle matematiskt kunna skrivas som $f(x) = x + x$. Notera att instruktionen **return** tydligt talar om att "här är resultatet". När man väl definierat funktionen, så kan den kallas på följande sätt:

```
f(2.0)
```

4.0

Lägg märke till att Python inte bryr sig om vilken typ argumentet har, så länge alla operationer och uttryck i funktionen går att genomföra. Ovan används ett flyttal, men vi kan också göra:

```
f(2)
f('hello')
```

```
4
'hellohello'
```

Precis som matematiska funktioner, så kan funktioner i Python ha flera argument:

```
def long_name(course_code, course_name):
    lname = course_code + course_name
    return lname
```

Vad är resultatet av `long_name('DA', '2005')`?

Funktioner behöver inte ta några argument

```
def show_user_menu():
    print('Menu:')
    print('1. Add user')
    print('2. List users')
    print('3. Quit')
```

och kan också anropa andra funktioner:

```
def user_choice():
    show_user_menu()
    return input('Which assignment do you want to do? ')
```

Notera att funktionsanrop alltid har parenteser efter funktionsnamnet, även när funktionen inte tar några argument.

2.5.1 Om funktioner

Exemplen ovan introducerar flera nya syntaktiska koncept i Python. En funktion har en definition (även kallad “kropp”) och ett huvud (även kallad “signatur”).

- Huvudet består av funktionsnamnet och en parameterlista (“funktionens argument”).
 - Syntax: `def funktionsnamn(argument1, argument2, ...)`:
- Ordet parameter har två betydelser.
 - Variabler i huvudet kallas *formella parametrar*. Formella parametrar är platshållare för de värden som kommer att ges vid ett funktionsanrop.
 - De värden man ger vid ett anrop till en funktion kallas *anropsparametrar* eller *argument*.
- Kroppen har en dokumentationssträng (frivillig) följt av kod, och är *indragen*.

Indragningen är viktig i Python! Python är ett “indenteringskänsligt” språk, dvs fel indrag ger syntaxfel. Vi använder här ordet *indentering* när vi menar indrag i källkoden. Indenteringen är det som specificerar det *kodblock* som hör till funktionskroppen. Även om man kan använda sig av vilken indentering man vill i Python (till exempel, två mellanslag eller en tab), så länge man är konsekvent, så är *standard-indenteringen* fyra mellanslag.

Att indenteringen ska vara fyra mellanslag specificeras i Pythons stilguide, känd som [PEP-8](#), som också innehåller mycket annan nyttig information om hur man skriver bra Python-kod.

En modern programmeringseditor sätter in fyra mellanslag när använder tab-tangenten. En liten varning dock för att äldre eller mer primitiva editorer kanske inte hanterar tab-tangenten på ett bra sätt. Det kan ge problem beroende på hur editorn är konfigurerad vilket gör att er kod kan se olika ut på olika datorer.

Funktionskroppen avslutas ofta med en **return**-instruktion, till exempel **return** x. Om **return** saknas så returneras värdet None. Det är bra att veta att **return** kan förekomma fler gånger i en funktion, men om en **return**-sats exekveras så avslutas alltid funktionen.

Det är viktigt att förstå skillnaden på funktioner som returnerar något och de som bara till exempel skriver ut något. Betrakta följande kodsnuitt:

```
def f1(x):  
    return x + x  
  
def f2(x):  
    print(x + x)
```

Skillnaden är att f1 *returnerar* värdet på x + x medan f2 bara *skriver ut* värdet. Om vi nu försöker skriva ut resultatet av f1(2) blir det som vi förväntat oss:

```
print(f1(2))
```

```
4
```

men om vi försöker skriva ut resultatet av f2(2) händer följande:

```
print(f2(2))
```

```
4
```

```
None
```

Anledningen är att 4 först skrivs ut av funktionen f2, men sen när vi skriver resultatet av f2 så skrivs None ut då det är vad f2 faktiskt returnerar! Det är väldigt viktigt att förstå den här skillnaden och när det står att ni ska skriva en funktion som *returnerar* något så ska **return** användas och inte **print**.

2.5.2 Multipla returvärden

Hur gör man om man vill returnera flera värden? Det finns flera dåliga sätt att lösa detta, men ett bra: multipla returvärden.

```
def integer_div(nominator, denominator):  
    q = nominator // denominator  
    r = nominator % denominator  
    return q, r
```

Denna funktion kan man sedan anropa så här:

```
quotient, remainder = integer_div(17, 10)
```

2.5.3 Standardvärden

Det är vanligt att man skriver funktioner med parametrar som nästan alltid sätts till samma värde. Python låter dig ange explicit sådana vanliga värden: dessa kallas *standardvärden* (eng.: *default values*) eller *skönsvärden*.

```
def f(x=1):  
    return x + x  
  
print(f())  
  
print(f(3))
```

```
2  
6
```

I första utskriften kan vi se att `f` har kallats med `x=1`, även om vi inte gett det som argument. När vi på nästa rad kallar `f(3)`, så sätts `x=3`.

Följande funktion testar om en punkt (x, y) ligger inom en cirkel med radie r . Standardvärdet `r=1` gör så att cirkeln är enhetscirkeln om man inte ger tredje argumentet.

```
def in_circle(x,y,r=1):  
    # Compute length of hypotenuse  
    h = (x ** 2 + y ** 2) ** 0.5  
  
    # Is the point within the radius?  
    return h <= r  
  
print(in_circle(1,1,2))  
print(in_circle(1,1))  
print(in_circle(0.5,0.5))
```

```
True  
False  
True
```

2.5.4 Nyckelordsparametrar

Python låter dig namnge parametrar även i anropen. Till exempel så ger `integer_div(denominator=10, nominator=17)` ger samma effekt som anropet `integer_div(17, 10)`.

Vad händer om vi gör:

```
integer_div(denominator=10, nominator=17)
```

Python kopplar identifierarna på de formella parametrarna till identifierarna på argumenten, så ovan resulterar i precis samma output.

Ett exempel på en funktion som vi använt som har fler nyckelordsparametrar är `print`. Den har följande nyckelordsparametrar och standardvärden:

- `sep=' '` — dvs ett mellanslag som separator
- `end='\n'` — “backslash-n” betyder radbrytning.

- `file=sys.stdout` — låt utdata gå till terminalen

Exempel:

```
print('hello', 'you', sep='\n')
```

```
hello
you
```

För att se mer information skriv `help(print)` i Spyder-konsolen (iPython).

2.5.5 Dokumentation av funktioner

Det finns två sätt att dokumentera funktioner i Python, *dokumentationssträngar* (eng: *docstrings*) och *kommentarer*. Dokumentationssträngar är den primära dokumentationsmetoden och består av en sträng överst i funktionskroppen. Kommentarer ges till stöd för framtida läsning av kod.

2.5.5.1 Dokumentationssträngar

En dokumentationssträng ska förklara hur en funktion används. En tumregel är att en annan programmerare ska förstå hur funktionen används utan att behöva studera dess kod.

Även om funktionen har ett beskrivande namn är det inte alltid tydligt vad den faktiskt gör. Om en funktion till exempel heter `compute_integral`, behöver dokumentationssträngen inte upprepa detta, men det kan vara värdefullt att beskriva vilken algoritm som används, vilka antaganden som görs om parametrarna och vad som gäller för returvärdet.

Här är ett exempel på en enkel enradig dokumentationssträng:

```
def square(a):
    """Returns argument a squared."""
    return a*a

print (square.__doc__)

help(square)
```

Av konvention använder man flerradssträngar med citattecken, dvs skriver `""" """`, oavsett om man använder en eller flera rader.

En lite mer ambitiös dokumentationssträng kan se ut så här:

```
def square(x):
    """Description of square function

    Parameters:
    x (int): input number

    Returns:
    int: Square of x
    """

    return x*x
```

```
print(square.__doc__) # This is how docstrings are "stored"

help(square) # Try this!
```

Det speciella med dokumentationssträngar är att de knyts till funktionen och blir automatiskt tillgänglig som 'officiell' dokumentation. Du kan se dokumentationen för all kod i Python-systemet, moduler som du installerar, och den kod du skriver själv med kommandot `help(identifier).{python}` (prova i Spyders console). Om du använder en bra kodeditor så blir dokumentationen tillgänglig som popup-rutor och annat. I Spyder gör Ctrl-i när markören står vid ett funktionsanrop att dess dokumentation visas.

Det finns ingen riktig standard för hur man ska skriva dokumentationssträngar. De är beskrivna så här i [PEP-257](#):

The docstring [...] should summarize its behavior and document its arguments, return value(s), side effects, exceptions raised, and restrictions on when it can be called (all if applicable). Optional arguments should be indicated. It should be documented whether keyword arguments are part of the interface.

Vissa projekt och företag väljer att specificera en slags grammatik för dokumentationen, men för din egen kod har du full frihet att välja struktur på dokumentationen.

2.5.5.2 Kommentarer dokumenterar implementation

I Python är kommentarer inte lika viktiga som i många andra programmeringsspråk. Språket är utformat för att likna "pseudokod", dvs algoritmiska beskrivningar, och många gånger (särskilt med väl valda identifierare) kan man gott säga att Python-kod är självförklarande. Naturligtvis kräver det att man har tänkt igenom hur koden presenteras, och det finns gott om tillfällen när det är bra att använda en kommentar för att förklara vissa designval eller konstiga kodrader.

Ett vanlig nybörjarfel är att man skriver kommentarer överallt, men man ska inte underskatta den som läser din kod. Det här är till exempel en dålig kommentar:

```
n += 1 # Add 1 to n
```

Den kommentaren är helt redundant och ger mer att läsa utan att tillföra något.

Se istället de här två kommentarerna. De förklarar varför man valt att begränsa antalet iterationer när man löser en ekvation $f(x) = 0$ numeriskt och varför man vågar dividera med x utan att verifiera att x inte är nära noll.

```
...
# 10 iterations provides sufficient precision when solving f(x) = 0
x = newton_raphson(f, start_value, maxiter=10)
return magic_value / x # We know that f(x) > 1, so safe to divide
```

2.6 Globala och lokala variabler

Variabler i Python kan vara antingen *globala* eller *lokala*. Namnet syftar till var en variabel är "synlig" för kod, och var variabeln kan modifieras/ändras. I följande exempel är x en global variabel då den har definierats på toppnivå (d.v.s. "längst ut").

```
x = "My global string"
```

```
def foo():
    print("x printed from inside foo:", x)

foo()
print("x printed from outside foo:", x)
```

Funktionen kan inte modifiera x, utan värdet som x innehåller är bara läsbart:

```
x = 42

def foo():
    x = x * 2
    print("x printed from inside foo:", x)

foo()
```

UnboundLocalError: local variable 'x' referenced before assignment

För att kunna ändra på värdet i x, så måste vi säga åt funktionen att den kan ändra på den globala variabeln. Det här gör man med det reserverade ordet **global**.

```
x = 42

def foo():
    global x
    x = x * 2
    print(x)

foo()
```

Globala variabler som ändras av funktioner på detta sätt kan lätt leda till mycket röriga program, det man ibland kallar "spagettikod". Det blir svårt att hålla reda på var värdet på x ändras, så var försiktig om du använder globala variabler på detta sätt. Det finns scenarion när globala variabler är användbara, exempelvis globala räknare. Det är då extra viktigt att ha ett informativt namn på den globala variabeln så att man inte råkar ändra den av misstag!

Lokala variabler är variabler som definieras inuti i en funktion:

```
def foo():
    y = "My local string"
    print(y)

foo()
```

Variabeln y syns inte utanför funktionen (det är därför det kallas för *lokala* variabler):

```
def foo():
    y = "local"

foo()
print(y)
```

NameError: name 'y' is not defined

Var x är synlig är dess *räckvidd* (eng. *scope*). Om x är definierat utanför funktionen och vi sedan skriver

över den i funktionen så är dess värde oförändrat utanför funktionen:

```
x = 5

def foo():
    x = 10
    print("Local x:", x)

foo()
print("Global x:", x)
```

Samma sak händer med funktionsargument:

```
x = 5

def foo(x):
    print("Local x:", x)

foo(12)
print("Global x:", x)
```

2.7 Villkorssatser

Det händer ofta att man vill exekvera ett visst kodblock endast om något uttryck eller något test är sant. Detta görs med så kallade *villkorssatser*, eller "if-satser". I Python görs detta med **if**, **elif** (som står för *else if*) och **else**.

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
print('Okay, I guess you were not finished yet.')
```

Obs: lägg märke till dubbla likhetstecken! Det är skillnad på *jämförelse* och *tilldelning* (dvs == och =).

Exemplet ovan kan även skrivas:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not finished yet.')
```

Notera att print-satsen är indragen lika långt som quit. Precis som för funktioner så gäller det att det indenterade kodblocket exekveras.

Indenteringsfel är svåra att se ibland:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not...')
    print('.. quite finished yet.') # indentation error
```

Ett till exempel med **else**:

```
namn = input('What is your name? ')
if namn == 'Anders':
    print('Hello Anders!')
else:
    print('Hello, whoever you are!')
```

Man kan använda sig av *nästlade* villkorssatser, dvs en villkorssats inuti en villkorssats:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
else:
    if temp < 0:
        print('That is pretty cold...')
    else:
        if temp == 0:
            print('Zero')
        else:
            if temp < 30:
                print('That is a comfortable temperature')
            else:
                print('Wow! Super warm')
```

I ovan fall är koden inte särskilt lättläst. Ett bättre alternativ är att använda **elif**:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
elif temp < 0:
    print('That is pretty cold...')
elif temp == 0:
    print('Zero')
elif temp < 30:
    print('That is a comfortable temperature')
else:
    print('Wow! Super warm')
```

Läsbarhet är viktigt! Det reserverade ordet **elif** är strikt talat ej nödvändigt då det alltid kan definieras med nästlade **if-else**, men koden blir lättare att läsa. Denna typ av funktionalitet kallas ibland *syntaktiskt socker*.

Vad är skillnaden på beteendet för

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
if x > 42:
    print('Too high...')
else:
```

```
print('Too low...')
```

och

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
elif x > 42:
    print('Too high...')
else:
    print('Too low...')
```

Vad händer om man ger 42 som svar till input?

Slutligen så kan vi använda oss av villkorssatser för att skriva om xor-uttrycket från [Logiska operatorer](#) och definiera en funktion:

```
def xor(x, y):
    if x and y:
        return False
    elif not x and not y:
        return False
    else:
        return True

print(xor(True, True))
print(xor(True, False))
print(xor(False, True))
print(xor(False, False))
```

```
False
True
True
False
```

En kortare version kan med det logiska uttryck vi skrev tidigare:

```
def xor(x, y):
    return not ((x and y) or (not x and not y))
```

Vilken version av funktionen xor är lättast att läsa?

2.8 Uppgifter

1. Vilket eller vilka av alternativen nedan ska man tänka på när man väljer ett namn på en funktion eller variabel?
 - a) Namnet ska vara kort
 - b) Namnet ska vara beskrivande
 - c) Namnet ska vara unikt i funktionen och/eller skriptet
 - d) Namnet ska vara skrivet med stora bokstäver
 - e) Inget av ovanstående

2. Skriv kod som läser in ett heltal från användare och skriver ut Odd om talet är udda och Even om talet är jämnt.
3. Gör sanningstabeller för de booleska operatorerna and och or (d.v.s. fyll i ? i tabellen nedan):

x	y	x and y	x or y
True	True	?	?
True	False	?	?
False	True	?	?
False	False	?	?

4. Försök, utan att köra koden, utvärdera dessa fyra uttryck för alla möjliga booleska värden på x, y och z:

```
(not x) or (not y)
x and (y or z)
(x != z) and not y
x and z
```

Kör sedan koden för att bekräfta dina svar.

5. Skriv en Python-funktion som motsvarar den matematiska funktionen $f(x, y) = x/y$. Vi vill att funktionen ska säga till om man delar ett tal skilt från 0 med 0. I detta fall så ska funktionen skriva ut `The result is infinite...`, men *inte* returnera något explicit värde. Om man delar 0/0, så ska funktionen skriva ut `Indeterminate form...` och inte heller returnera något.

6. Låt:

```
x = True
y = False
```

Vilka uttryck nedan får värdet `True`? Varför?

```
(x and y) or (not x and not y)
(x or y) and (not x or not y)
x or y or not x or not y
not(not x and not (x and y))
```

7. Definiera nand, nor och xnor som Python-funktioner.

Tips: https://en.wikipedia.org/wiki/Logic_gate#Symbols.

8. Prova funktionerna nand, nor och xnor med argument som *inte* är av typen bool. Är resultaten som du väntade dig? Om inte, varför?